

MySQL WHERE Clause Conditions

A Comprehensive Technical Reference Guide for Data Filtering

The `WHERE` clause is an indispensable component of MySQL query execution, utilized to extract only those records that fulfill specific criteria. It filters rows vertically during execution before any grouping or projections take place, optimizing memory usage and performance.

1. Comparison Operators

Comparison operators form the core foundation of data filtering, allowing direct comparison of column values against scalar expressions, constants, or other column definitions.

Operator	Description	Example Syntax
<code>=</code>	Equal to	<code>SELECT * FROM employees WHERE department = 'Engineering';</code>
<code><></code> or <code>!=</code>	Not equal to	<code>SELECT * FROM products WHERE status != 'Discontinued';</code>
<code>></code>	Greater than	<code>SELECT * FROM orders WHERE total_amount > 500.00;</code>
<code><</code>	Less than	<code>SELECT * FROM stock WHERE quantity < 10;</code>
<code>>=</code>	Greater than or equal to	<code>SELECT * FROM users WHERE age >= 21;</code>
<code><=</code>	Less than or equal to	<code>SELECT * FROM invoices WHERE due_date <= CURDATE();</code>
<code><=></code>	NULL-safe equal to	<code>SELECT * FROM users WHERE supervisor_id <=> NULL;</code>

Note on the `<=>` Operator: Unlike the standard `=` operator which returns `NULL` when evaluating comparisons against missing values, the NULL-safe equal operator returns `1` (True) if both operands are `NULL` and `0` (False) if one operand is `NULL`.

2. Logical Operators

Logical operators allow multiple evaluation conditions to be combined seamlessly inside a single query block.

Operator	Description	Example Syntax
<code>AND</code>	Evaluates true if all joined conditions are true	<code>WHERE role = 'Manager' AND clear_level > 3</code>
<code>OR</code>	Evaluates true if any joined condition is true	<code>WHERE region = 'EMEA' OR region = 'APAC'</code>
<code>NOT</code> / <code>!</code>	Inverts the logic of the condition	<code>WHERE NOT (status = 'Archived' OR stock = 0)</code>

3. Range & Set Matching Operators

When dealing with bounded bounds or explicit arrays of categorical structures, specialized matching expressions provide cleaner structures than repetitive comparison syntax.

BETWEEN ... AND

Filters rows falling inclusively within a minimum and maximum boundary limit. It handles integers, floating points, and datetime objects natively.

```
SELECT * FROM transactions
WHERE transaction_date BETWEEN '2026-01-01' AND '2026-03-31';
```

IN (...)

Verifies if a target scalar value is equal to any element specified inside a comma-separated explicit array or returned subquery list.

```
SELECT * FROM inventory
WHERE supplier_id IN (101, 104, 108, 215);
```

4. Pattern & String Matching Operators

For flexible string search operations and unstructured structural texts, MySQL provides wildcards and sophisticated system matching engines.

LIKE

Evaluates basic wildcard expressions. The percentage character (`%`) matches zero or more sequence strings, while the underscore symbol (`_`) handles exactly one single character frame.

```
-- Finds all emails ending with a standard gmail structural domain
SELECT * FROM clients WHERE email LIKE '%@gmail.com';

-- Finds names exactly four characters long starting with 'A'
SELECT * FROM profiles WHERE first_name LIKE 'A___';
```

REGEXP / RLIKE

Evaluates complex POSIX extended regular expressions to offer extreme control over pattern-matching operations.

```
-- Filters alphanumeric strings beginning with 2 alphabetic letters followed by numbers
SELECT * FROM vouchers WHERE voucher_code REGEXP '^[A-Z]{2}[0-9]+$';
```

5. Missing Data / Null Evaluation

Because structural missing fields evaluate as an unknown element value, traditional comparisons fail against them. Dedicated unary constructs must be evaluated explicitly.

Condition	Purpose	Example Syntax
<code>IS NULL</code>	Filters explicitly missing entries	<code>WHERE termination_date IS NULL</code>
<code>IS NOT NULL</code>	Filters records containing a set value	<code>WHERE alternative_phone IS NOT NULL</code>

6. Subquery Evaluation Expressions

Subqueries can be embedded dynamically inside `WHERE` definitions to correlate criteria values across multiple interdependent relational database entities.

- **`EXISTS (subquery)`** : Evaluates to True if the internal subquery safely fetches at least one matching layout row.
- **`ANY / SOME (subquery)`** : Evaluates to True if comparison evaluates as True against any row produced in the subquery array.
- **`ALL (subquery)`** : Evaluates to True if comparison matches successfully against all independent records evaluated inside the nested execution.

```
-- Finds customers who have placed at least one high-value order
SELECT * FROM customers c
WHERE EXISTS (
  SELECT 1 FROM orders o
  WHERE o.customer_id = c.id AND o.grand_total > 10000
);
```